University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Mobile Robot

Srikanth Nanda

12. Dezember 2025

1 System Overview

2 Jetson OS Installation

3 Part Details: Compute and Cameras

4 Part Details: Mechanics

5 Part Details: Electronics

6 Hardware Setup and Wiring

7 Calibration and Odometry

8 ROS2 Integration

9 Code Snippets and Tests

10 Quellen

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

System Overview

Mobile Robot Architecture I

- Compute: NVIDIA Jetson Nano Developer Kit
- Cameras: Raspberry Pi HQ / CS / V2
- Locomotion: 4× TT motors + metal caster
- Control: Qwiic motor driver
- Feedback: Qwiic OLED status display
- Communication: 433 MHz antenna
- Wiring: Qwiic cable ecosystem

Mobile Robot Architecture II

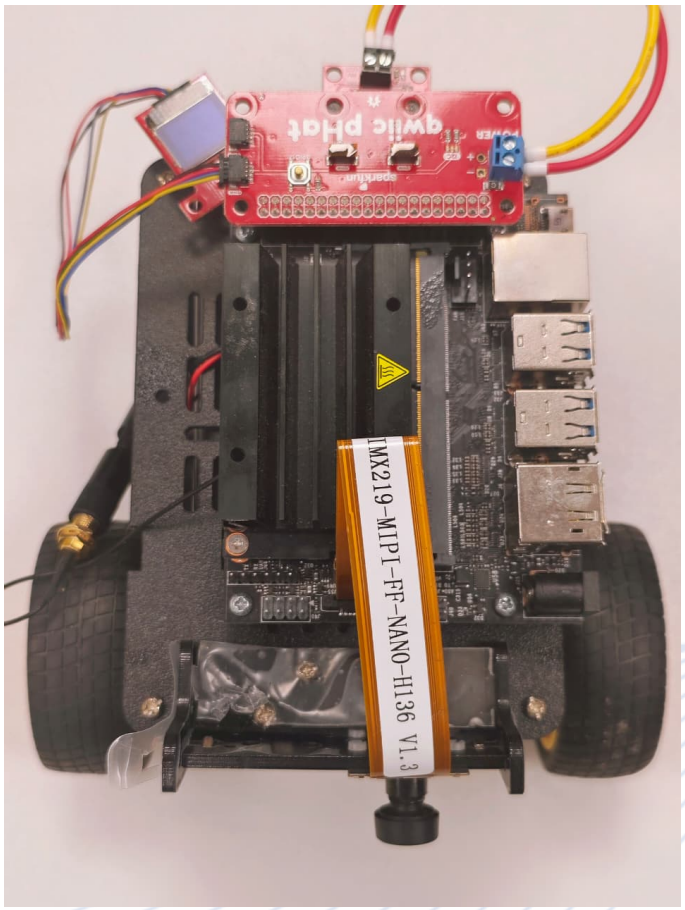
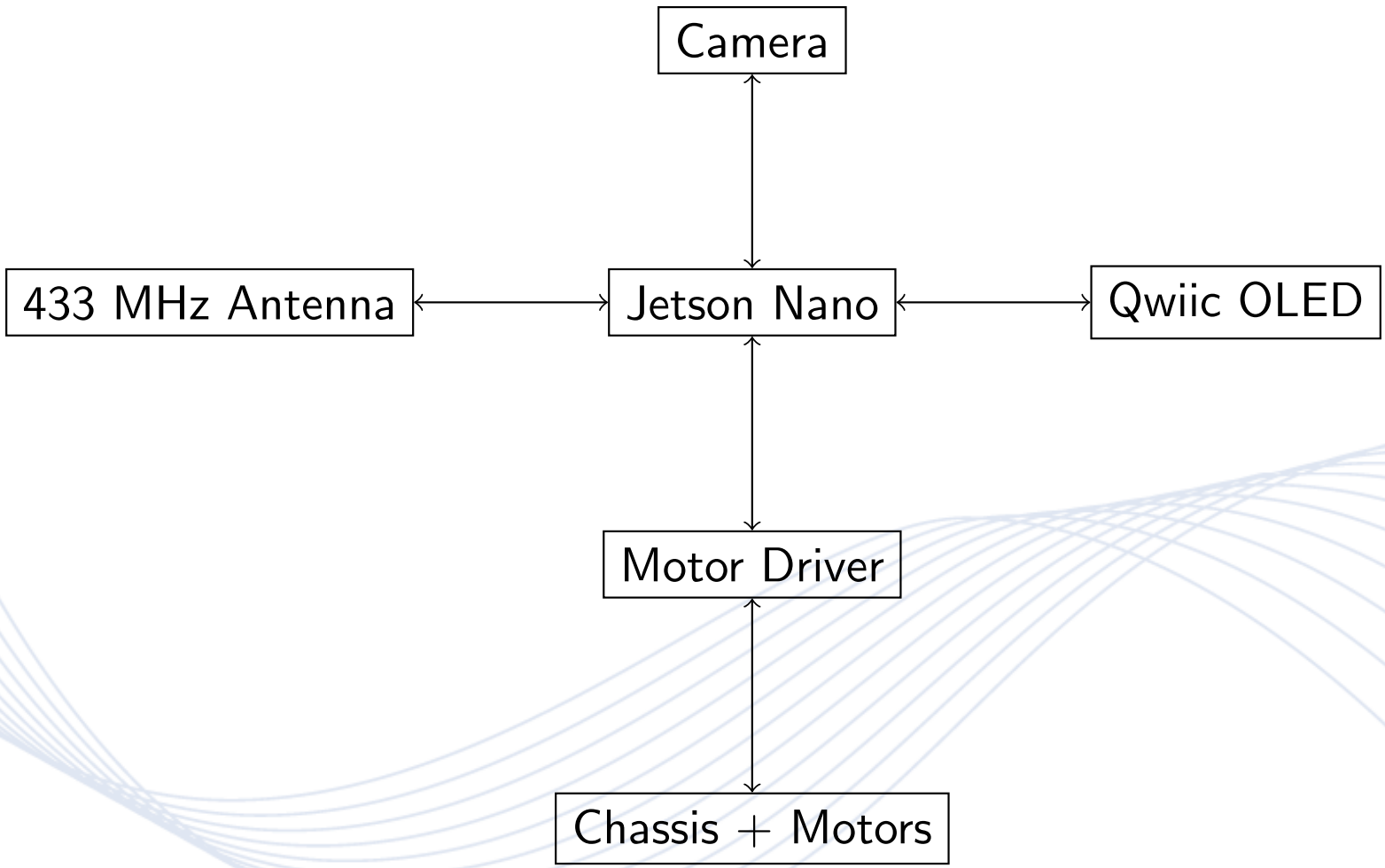


Abbildung: Mobile Robot Top View

Mobile Robot I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Jetson OS Installation

Install JetPack on Jetson I

Compute module: Jetson 10-1 A0 (Jetson Nano). **Steps:**

- Install NVIDIA SDK Manager on a Linux PC
- Select Jetson Nano target and JetPack version
- Flash image to SD card (using SDK Manager)
- Insert SD card into Jetson, connect power, HDMI, keyboard
- On first boot: complete Ubuntu setup and enable networking

Robot software stack:

- Install Python 3, OpenCV, and ROS2 (e.g. Humble) on Jetson
- Clone project repository containing `MR.py`

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Part Details: Compute and Cameras

Compute: Jetson Nano I

NVIDIA Jetson Nano Developer Kit(J1010): **Key features:**

- Quad-core ARM CPU + NVIDIA GPU for vision
- CSI ports for camera modules
- 40-pin header for GPIO / I^2C / UART
- Boots Ubuntu with JetPack

Role in robot:

- Runs MR.py and ROS2 nodes
- Processes images and computes odometry
- Sends motor commands and reads sensors via Qwiic

Leopard Imaging LI-IMX219-MIPI-FF-NANO Camera I

- **Module Part Number:** LI-IMX219-MIPI-FF-NANO
- **Compatible Platform:** NVIDIA Jetson Nano Developer Kit
- **Module Size:**
  1. H90: 150.0 mm (L) × 25.0 mm (W) × 15.27 mm (H)
  2. H136: 150.0 mm (L) × 25.0 mm (W) × 15.3 mm (H)
  3. H145: 150.0 mm (L) × 25.0 mm (W) × 14.1 mm (H)
- **Sensor Type:** Sony IMX219, 8.08 MP color
- **Active Pixels:** 3280 (H) × 2464 (V)
- **Optical Format / Diagonal:** Type 1/4.0, 4.60 mm
- **Aperture (F/No):**
  1. H90: F/2.6
  2. H136: F/2.0
  3. H145: F/2.5
- **Focal Length:**
  1. H90: 2.3 mm
  2. H136: 1.58 mm
  3. H145: 1.55 mm

Leopard Imaging LI-IMX219-MIPI-FF-NANO Camera I

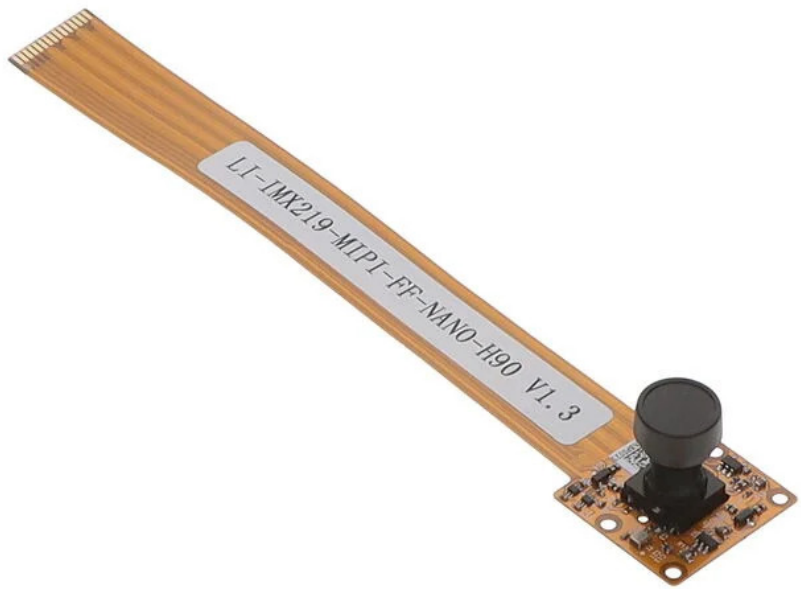


Abbildung: Leopard Imaging LI-IMX219-MIPI-FF-NANO Camera

Other Specifications of Leopard Imaging LI-IMX219-MIPI-FF-NANO Camera I

- **Horizontal Field of View (FOV):** 90°, 136°, 145°
- **TV Distortion:** ¡ -10
- **Focusing Range:** 30 cm to infinity
- **Pixel Size:** 1.12 µm × 1.12 µm
- **IR Cut Filter:** Yes
- **Operating Temperature:** -20°C to +60°C
- **Storage Temperature:** -30°C to +80°C
- **Data Output Interface:** CSI-2 MIPI (2-lane)
- **Maximum Image Transfer Rate:** 3280 × 2464 @ 21 fps
- **Mating Connector:** Part No. 1-1734248-5, 15 positions, 1.00 mm pitch

Raspberry Pi HQ Camera I

Camera: 12MP HQ Camera Lenses:

- 8–50 mm C-mount zoom lens
- HQ Camera USB Webcam Bundle

Raspberry Pi HQ Camera II



Abbildung: CCTVLENS 3MP Camera Mount

Raspberry Pi CS Camera I

Camera: 1.6MP global shutter C/CS camera Lens: 16 mm telephoto lens

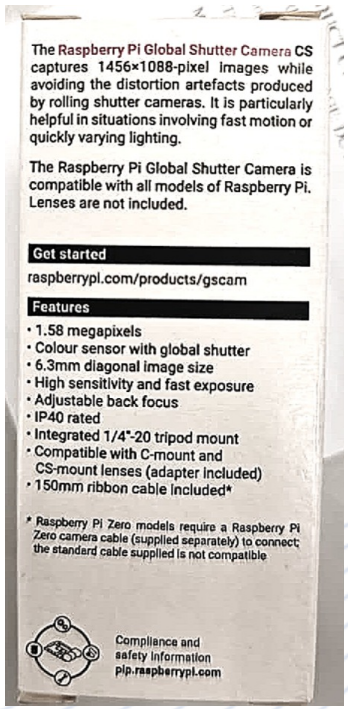


Abbildung: Raspberry CamGS Specifications

Raspberry Pi CS Camera II



Abbildung: Raspberry Pi Cam GS Cover

[1]

[1] Details:

- Global shutter for fast motion scenes
- Telephoto lens for distant feature tracking
- Same CSI connection method to Jetson

Raspberry Pi Camera V2 I

Camera: 8MP Camera V2 Flex cable: 15 cm flex cable



Abbildung: Raspberry Pi Cam V2 8MP

Raspberry Pi Camera V2 II

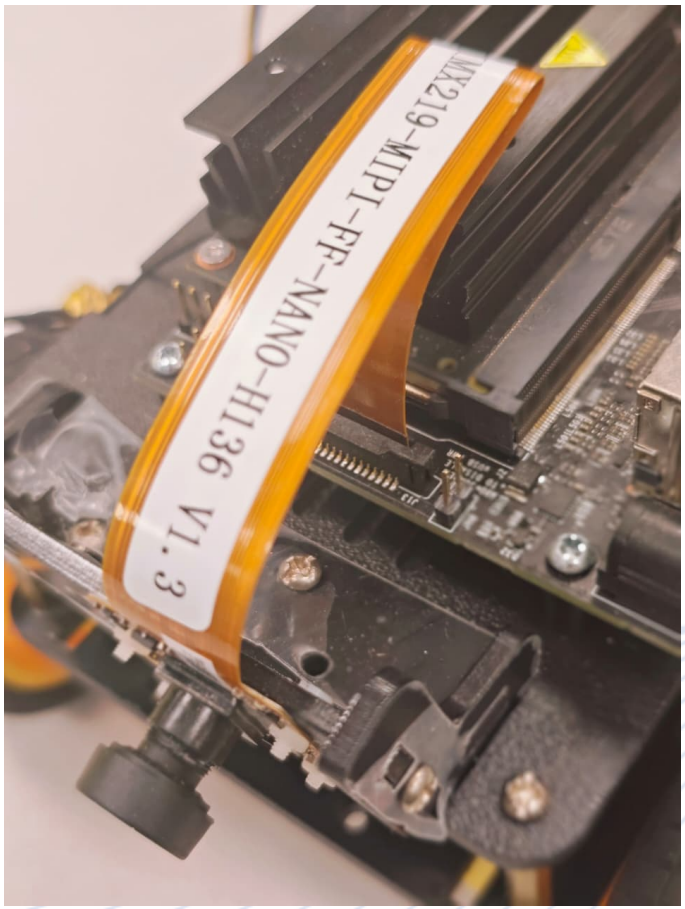


Abbildung: Flexible Cable for Camera

Raspberry Pi Camera V2 III

Details:

- Lightweight, compact module
- Good for navigation view
- Mounted with flex cable to front of chassis

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Part Details: Mechanics

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Chassis and Wheels I

Main chassis: Waveshare 2x TT motor chassis
Contents:

- Metal base plate
- 2× TT DC motors with wheels
- Shock-absorbing mounting for rough floors

Chassis and Wheels II

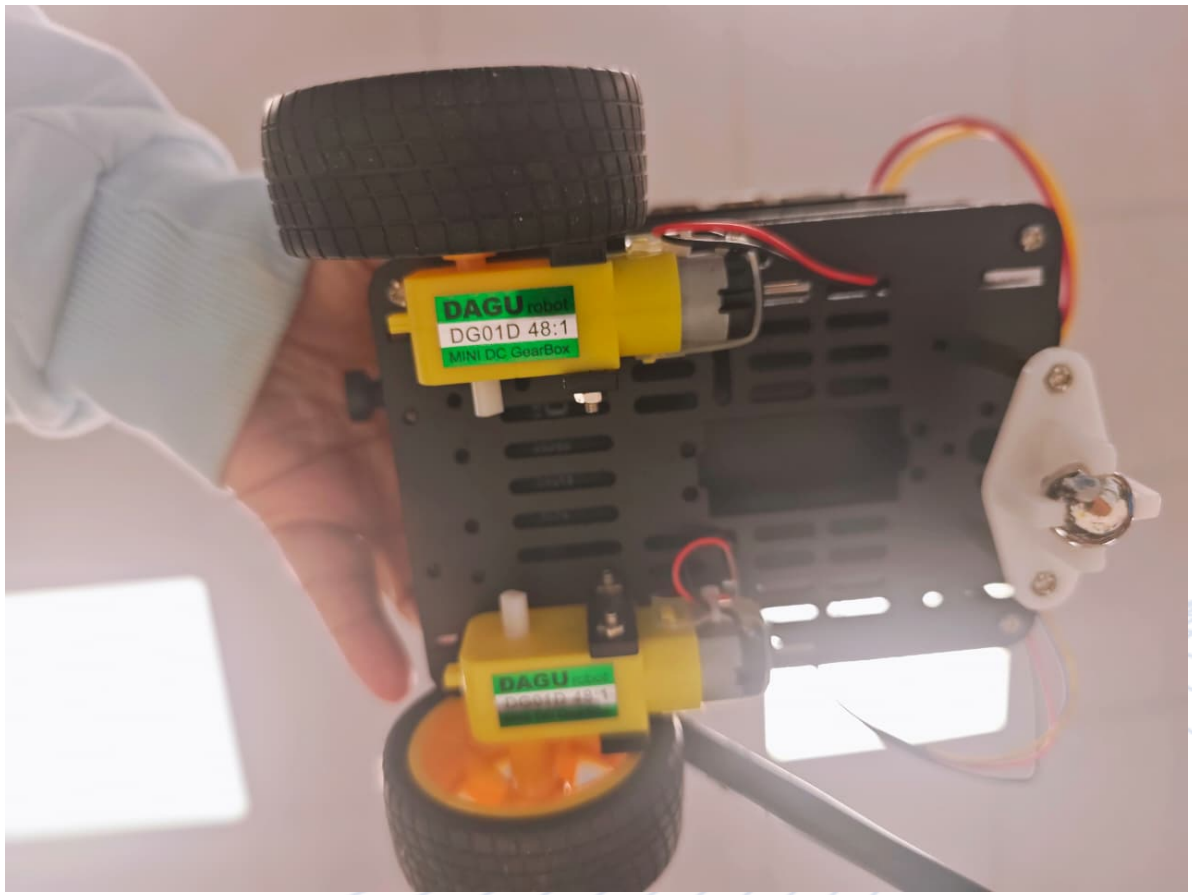


Abbildung: Mobile Robot Chassis

Chassis and Wheels III

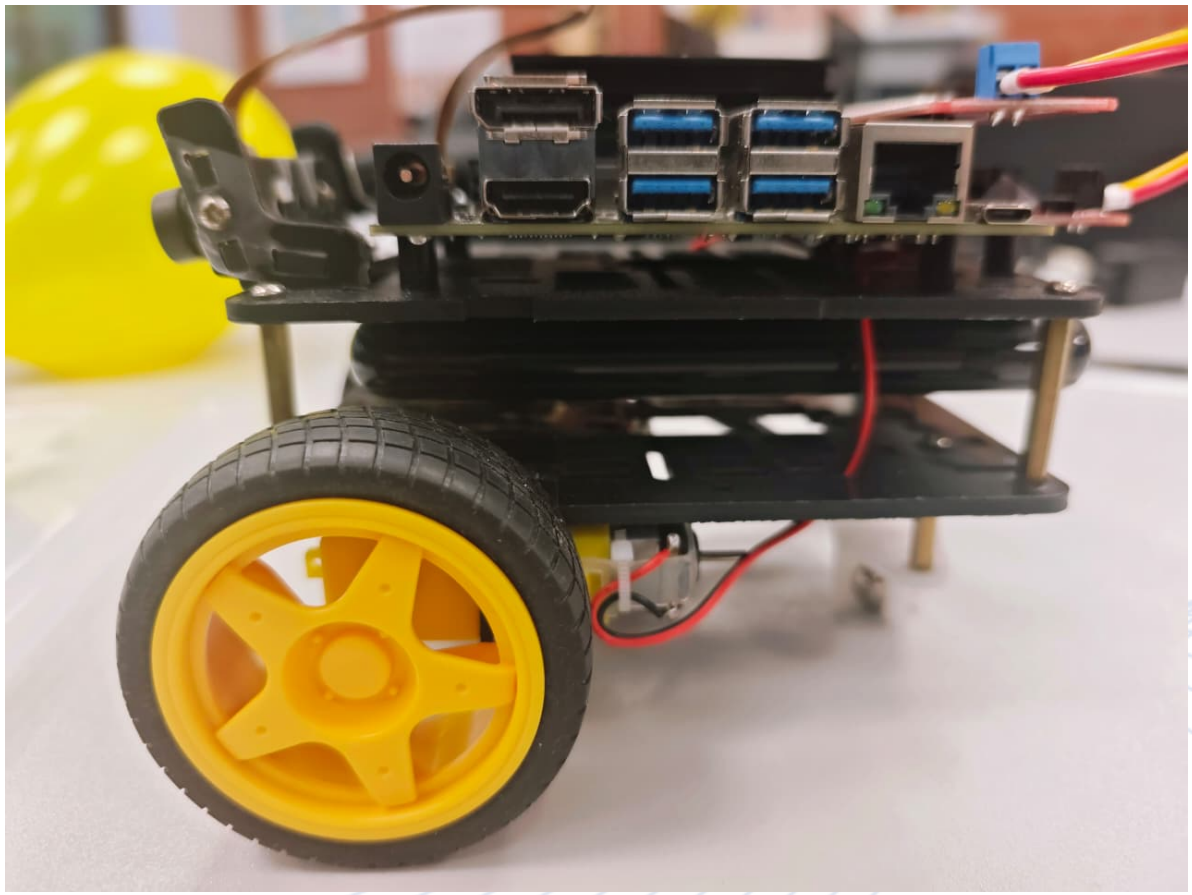


Abbildung: Mobile Robot Side View

Chassis and Wheels IV

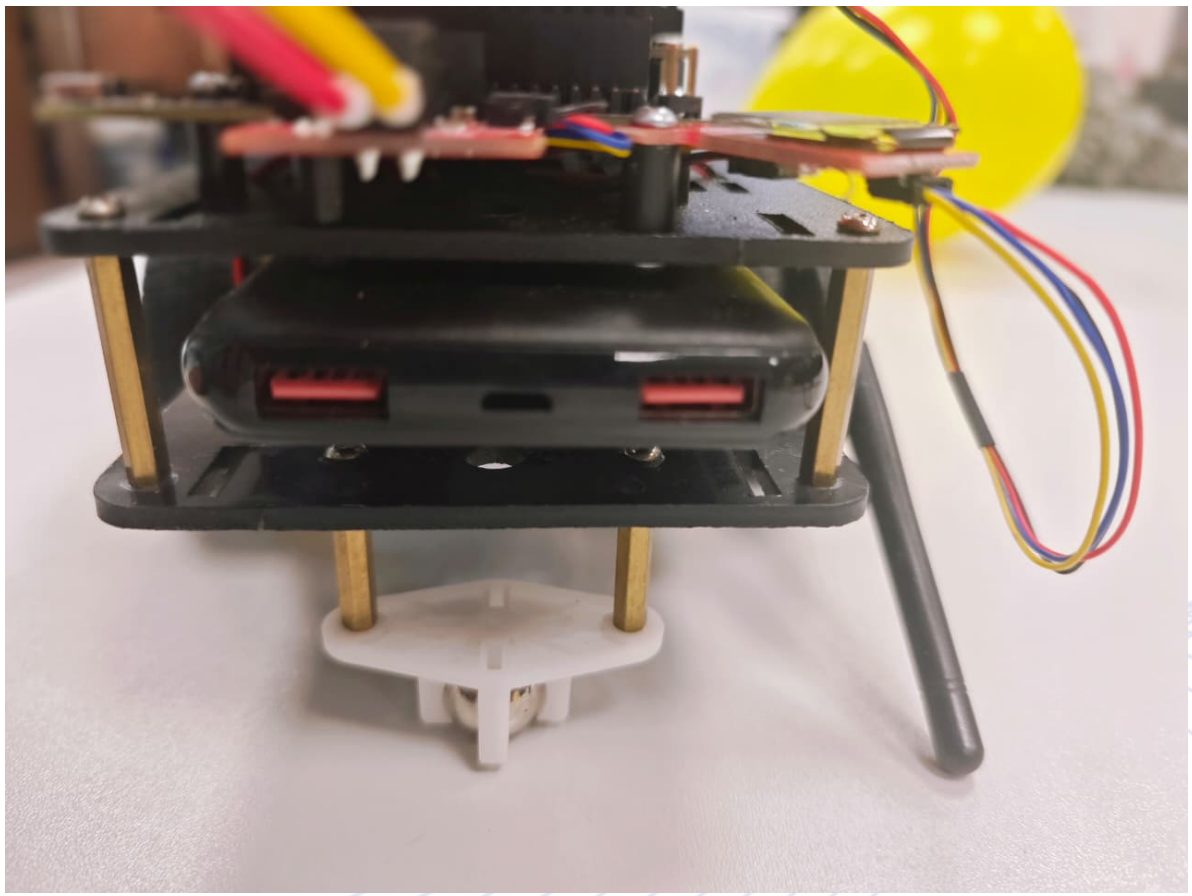


Abbildung: Mobile Robot Rear View

Chassis and Wheels V

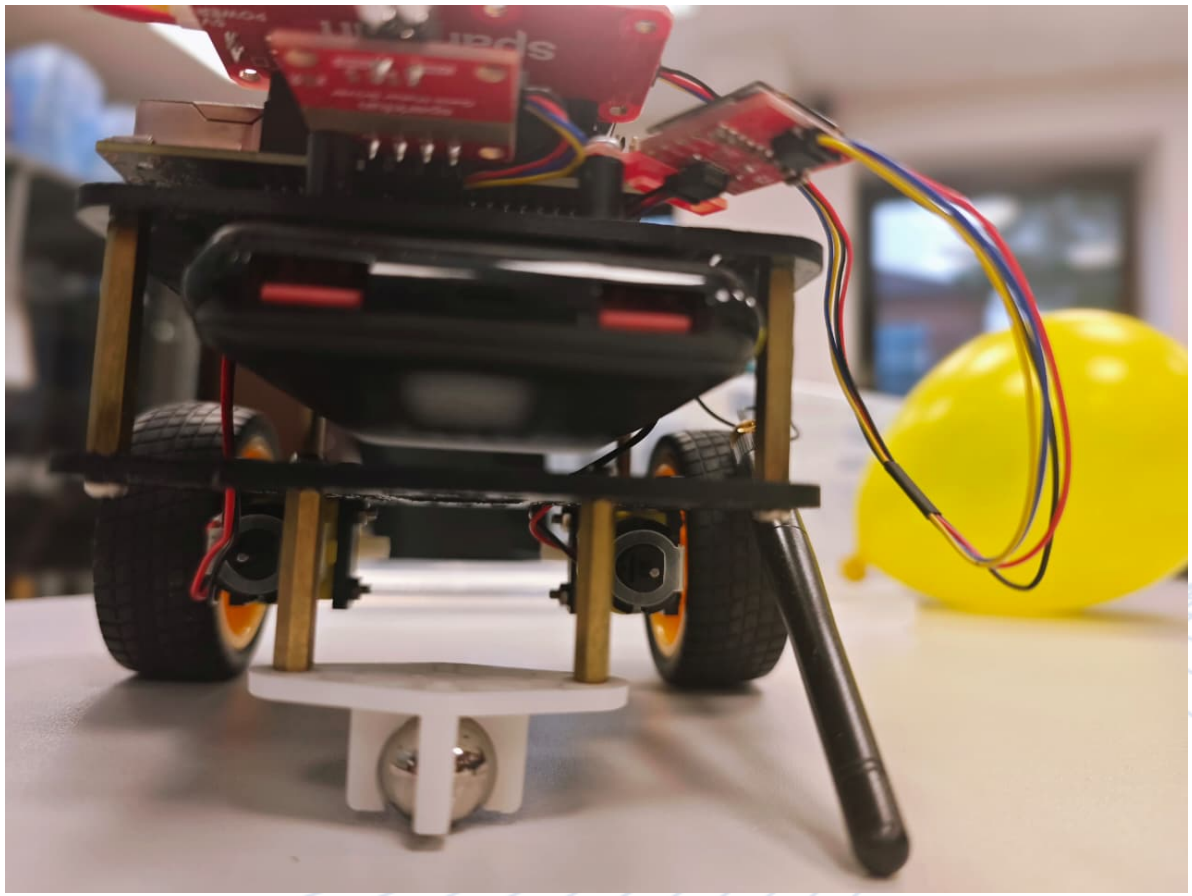


Abbildung: Mobile Robol Metal Caster Bearing 15.7mm

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Chassis and Wheels VI

Caster wheel: Adafruit metal caster stabilizes the rear of the robot.

Assembly Notes I

- Mount motors at the four corners using provided brackets
- Attach caster wheel at rear center for balance
- Position Jetson in the middle for weight distribution
- Mount cameras at the front for a clear field of view

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Part Details: Electronics

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Qwiic Boards I

Qwiic pHAT: SparkFun Qwiic pHAT Provides:

- Qwiic connectors for I^2C
- Clean breakout from 40-pin header

Qwiic Boards II

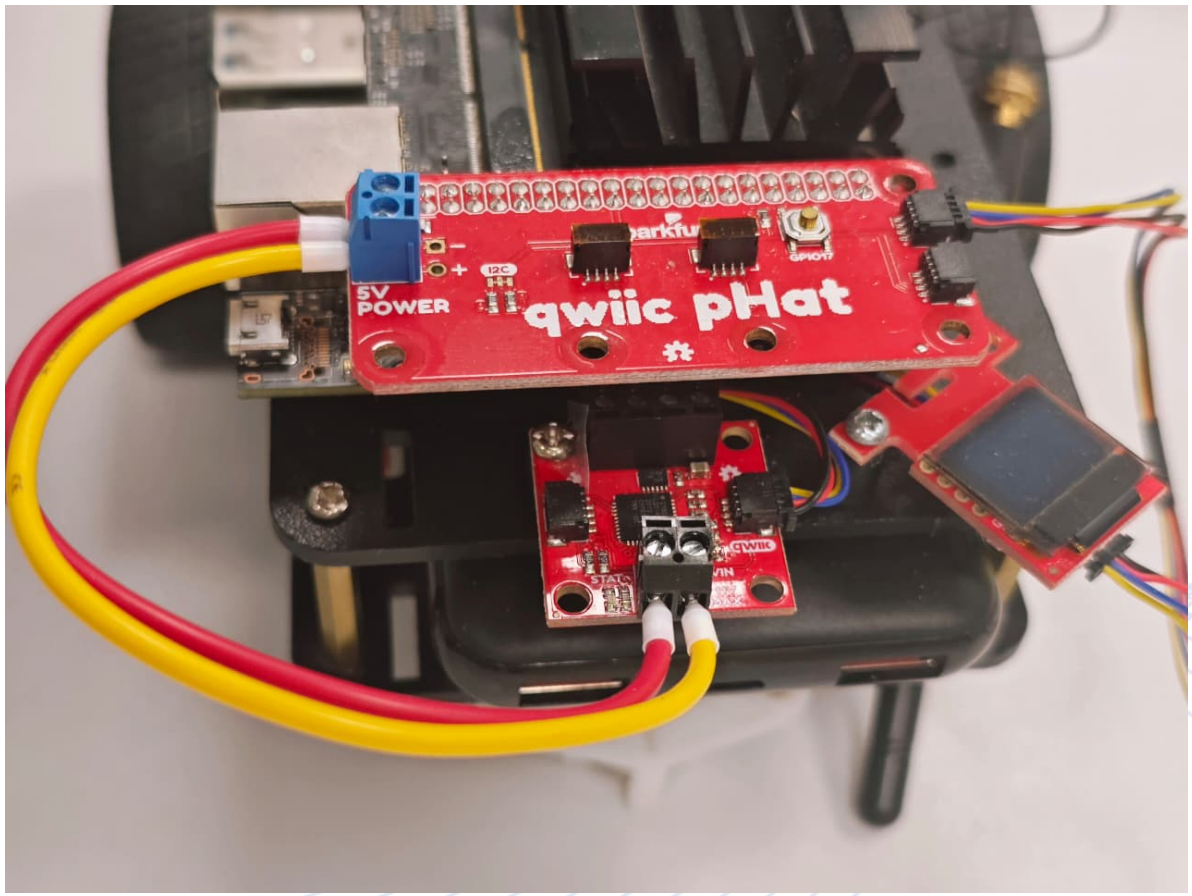


Abbildung: Mobile Robot Qwiic Dev Board

Qwiic Boards III

Qwiic Motor Driver: SparkFun Qwiic Motor Driver

- Controls DC motors via I^2C
- Power input for motor supply

Qwiic OLED Display: SparkFun Micro OLED

- Displays status (battery, mode, errors)
- I^2C-addressable

Cables and Antenna I

Qwiic Cables: SparkFun Qwiic Cable Kit

- Different lengths for flexible routing
- Daisy-chain multiple I^2C devices

Cables and Antenna II

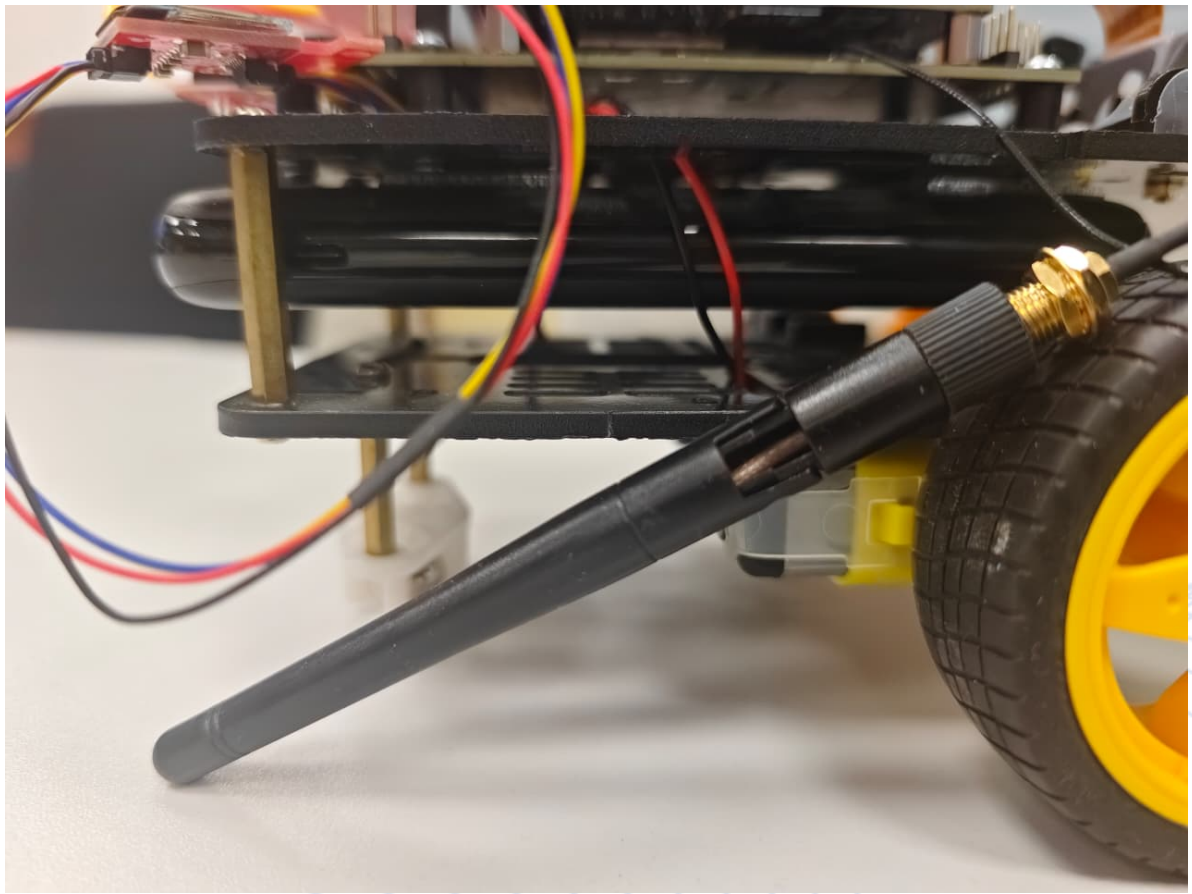


Abbildung: Mobile Robot Antenna

Cables and Antenna III

433 MHz Antenna: 433 MHz SMA antenna

- Connects to RF module on the robot
- Enables telemetry and remote control

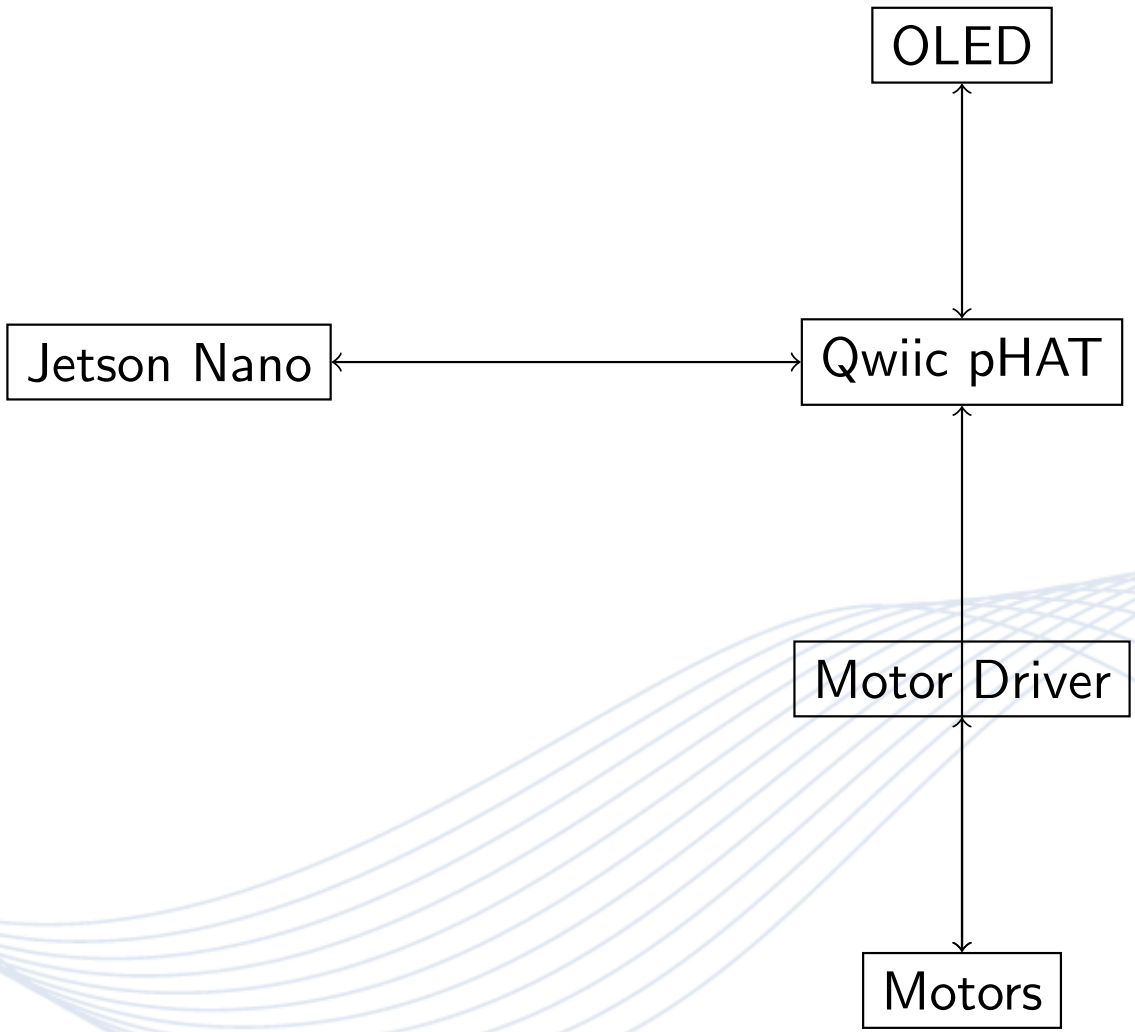
University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Hardware Setup and Wiring

Connections Overview I

- Jetson CSI → camera ribbon
- Jetson 40-pin header → Qwiic pHAT
- Qwiic pHAT → motor driver, OLED via Qwiic cables
- Motor driver terminals → TT motors on chassis
- RF module → 433 MHz antenna via SMA

Connections Overview Figure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Calibration and Odometry

Camera Calibration I

Goal: correct lens distortion and obtain camera matrix.

- Print a chessboard target and mount it on a flat surface
- Capture 20–30 images with different viewpoints
- Use an OpenCV calibration script offline
- Save intrinsic matrix and distortion coefficients for use in MR.py

Wheel Odometry I

Parameters:

- Wheel radius
- Wheelbase (distance between left and right wheels)

Procedure:

- Command robot to drive straight for a known distance
- Adjust scaling between motor command and wheel speed
- Turn robot in place by 360° and re-center heading

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

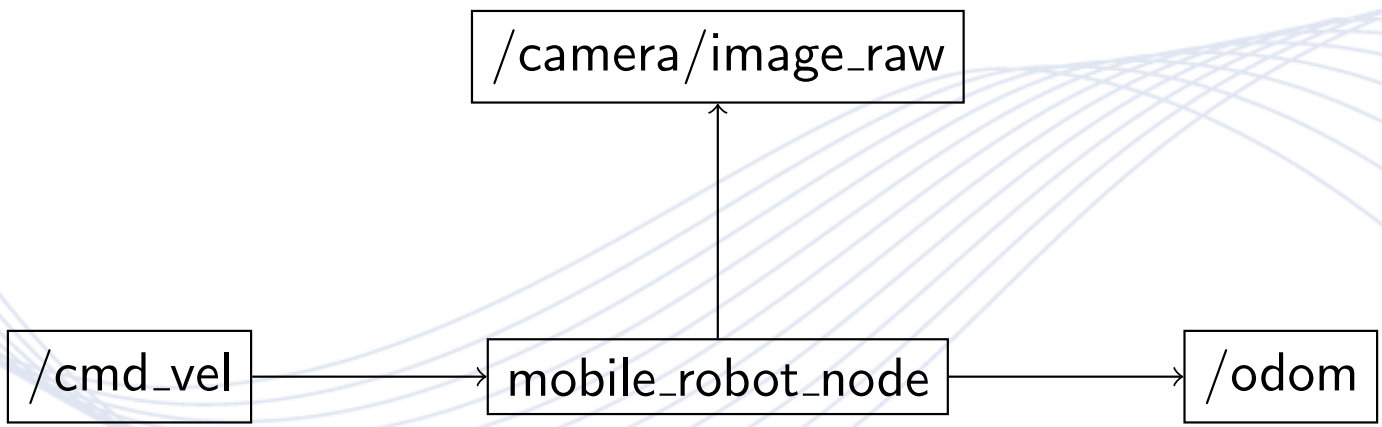
ROS2 Integration

ROS2 Node Graph I

In ROS2, MR.py can be wrapped in a node:

- **Node:** `mobile_robot_node`
- **Subscribes:** `/cmd_vel` (geometry_msgs/Twist)
- **Publishes:** `/odom` (nav_msgs/Odometry)
- **Publishes:** `/camera/image_raw` if desired

Conceptual layout:



ROS2 Behavior I

Inside ROS2 node:

- Timer callback runs odometry updates
- Subscriber callback updates target wheel speeds
- Odometry is published periodically on `/odom`
- Motor driver is commanded based on velocity commands

In MR.py we implement:

- A ROS2 `Node` class wrapping MotorDriver and Odometry
- A main entry function `run_ros2()` that starts the node

Code Snippets and Tests

Overview of MR.py

- Python3 script for controlling a Jetson-based differential-drive robot.
- Supports both:
  - **Standalone mode** (no ROS2 required)
  - **ROS2 mode** (`cmd_vel + odometry`)
- Components handled:
  - Camera initialization and capture
  - Motor driver abstraction
  - Differential-drive odometry
  - Calibration utilities
  - Test routines

Camera System

- Uses OpenCV's `VideoCapture`.
- Function: `init_camera(index, width, height)`.
- Steps:
  - Opens camera at given index.
  - Sets resolution (default 1280×720).
  - Returns OpenCV capture object.
- Warning printed if the camera fails to open.

Motor Driver Abstraction

- Class: `MotorDriver`.
- Placeholder for Qwiic motor driver (replace later with real hardware API).
- Attributes:
  - `left_speed`
  - `right_speed`
- Methods:
  - `set_speeds(left, right)` – prints motor command.
  - `stop()` – sets both motors to zero.

Odometry System

- Class: `Odometry`.
- Implements differential-drive kinematics.
- Parameters:
  - Wheel radius $r = 0.033$ m
  - Wheel base $L = 0.17$ m
- Tracks robot pose:
  - x position
  - y position
  - θ heading angle
- Updates via:

$$v = \frac{r}{2}(w_r + w_l), \quad \omega = \frac{r}{L}(w_r - w_l)$$

Calibration Routines

- **Camera Calibration** (placeholder)
  - Uses chessboard images in actual implementation.
  - Returns identity matrix $K = I$ and zero distortion.
- **Motor Calibration**
  - Runs both motors forward briefly.
  - Helps verify wiring and direction.

Test Functions

- `test_motors()`
  - Sweeps motors at speeds: 30, 60, 100, 0.
- `test_camera()`
  - Captures one test frame.
  - Saves as `test_frame.jpg`.

Standalone Mode

- Does not require ROS2.
- Workflow:
  - Initialize camera + motor driver + odometry.
  - Perform calibration tests.
  - Drive forward with fixed speed.
  - Print odometry for 20 iterations.
- Useful for basic debugging and hardware bring-up.

ROS2 Mobile Robot Node

- Class: `MobileRobotNode(Node)`.
- Subscribes to:
  - `/cmd_vel` (linear + angular velocity)
- Publishes:
  - `/odom` – robot pose and orientation
- Timer callback at 20 Hz:
  - Computes wheel velocities.
  - Updates odometry.
  - Converts yaw to quaternion.
  - Sends motor commands.

Entry Point

- Execution controlled by flag: `USE_ROS2`.
- If **True**: launches ROS2 node.
- If **False**: runs standalone test mode.
- Allows easy switching between development and deployment.

Code from MR.py I

```
#!/usr/bin/env python3
"""
/**
 * @file MR.py
 * @brief Mobile Robot Control System (Camera + Motors +
    Odometry + ROS2)
 * @author Srikanth
 * @date 2025
 *
 * This script controls a Jetson-based mobile robot with:
 *   - Qwiic motor driver (abstracted here)
 *   - Wheel odometry
 *   - Camera capture
 *   - Calibration hooks
 *   - Testing utilities
 *   - Optional ROS2 node interface (cmd_vel / odom)
 */
"""

import time
```

Code from MR.py II

```
20 import math
21 import cv2
22 import numpy as np
23
24 # ROS2 imports are optional; code can still run in
       standalone mode
25 try:
26     import rclpy
27     from rclpy.node import Node
28     from geometry_msgs.msg import Twist
29     from nav_msgs.msg import Odometry
30     from builtin_interfaces.msg import Time as RosTime
31     ROS2_AVAILABLE = True
32 except ImportError:
33     ROS2_AVAILABLE = False
34
35
36 #
       ------------------------------------------------------------
```

Code from MR.py III

```
37 # CAMERA SETUP
38 #
       ----------------------------------------------------------------

39 def init_camera(index=0, width=1280, height=720):
40     """Initialize a simple video capture for Jetson."""
41     cam = cv2.VideoCapture(index)
42     cam.set(cv2.CAP_PROP_FRAME_WIDTH, width)
43     cam.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
44     if not cam.isOpened():
45         print("[Camera] Warning: could not open camera index
       ", index)
46     return cam
47
48
49 #
       ----------------------------------------------------------------

50 # MOTOR DRIVER MOCKUP
```

Code from MR.py IV

```
51 #
       ------------------------------------------------------------------------

52 class MotorDriver:
53     """
54     Simple motor driver placeholder.
55     Replace with a real Qwiic Motor Driver API.
56     """
57
58     def __init__(self):
59         self.left_speed = 0.0
60         self.right_speed = 0.0
61
62     def set_speeds(self, left, right):
63         """
64         Set motor speeds (for now just prints; in real code,
        send to driver).
65         """
66         self.left_speed = left
67         self.right_speed = right
```

Code from MR.py V

```
68          print(f"[Motor] Left={left:.2f}, Right={right:.2f}")
69
70      def stop(self):
71          """Stop both motors."""
72          self.set_speeds(0.0, 0.0)
73
74
75  #   ------------------------------------------------------------
76  # ODOMETRY
77  #   ------------------------------------------------------------
78  class Odometry:
79      """
80      Differential-drive odometry model.
81      """
82
83      def __init__(self, wheel_radius=0.033, wheel_base=0.17):
```

Code from MR.py VI

```
84          self.x = 0.0
85          self.y = 0.0
86          self.theta = 0.0
87          self.r = wheel_radius
88          self.L = wheel_base
89
90      def update(self, wl, wr, dt):
91          """
92          Update pose given left/right angular velocities (rad
    /s).
93          """
94          v = self.r * (wr + wl) / 2.0
95          w = self.r * (wr - wl) / self.L
96
97          self.x += v * math.cos(self.theta) * dt
98          self.y += v * math.sin(self.theta) * dt
99          self.theta += w * dt
100
101         return self.x, self.y, self.theta
102
```

Code from MR.py VII

```
103
104 #
    ------------------------------------------------------------------
105 # CALIBRATION ROUTINES (PLACEHOLDERS)
106 #
    ------------------------------------------------------------------
107 def camera_calibration(image_paths):
108     """
109     Camera calibration placeholder.
110     """
111     print("[Calibration] Running camera calibration on image
        set...")
112     # Real implementation would use cv2.
    findChessboardCorners, etc.
113     K = np.eye(3)
114     D = np.zeros(5)
115     print("[Calibration] Using identity K and zero
    distortion as placeholder.")
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Code from MR.py VIII

```
116     return {"K": K, "D": D}
117
118
119 def motor_calibration(motor):
120     """
121     Simple calibration hook: run motors forward briefly.
122     """
123     print("[Calibration] Motor calibration: forward test..."
        )
124     motor.set_speeds(50.0, 50.0)
125     time.sleep(1.0)
126     motor.stop()
127     print("[Calibration] Motor calibration: done.")
128
129
130 #
        ----------------------------------------------------------------
131 # TEST FUNCTIONS
```

Code from MR.py IX

```
132 #
        ----------------------------------------------------------------------

133 def test_motors(motor):
134     """
135     Sweep motor speeds to verify wiring and direction.
136     """
137     print("[Test] Sweeping motor speeds...")
138     for s in [30, 60, 100, 0]:
139         motor.set_speeds(s, s)
140         time.sleep(0.5)
141     motor.stop()
142     print("[Test] Motor test complete.")
143
144
145 def test_camera(camera):
146     """
147     Capture a single test frame and save to disk.
148     """
149     print("[Test] Capturing one test frame from camera...")
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Code from MR.py X

```
150     ret, frame = camera.read()
151     if ret:
152         cv2.imwrite("test_frame.jpg", frame)
153         print("[Test] Saved test_frame.jpg")
154     else:
155         print("[Test] Failed to capture frame.")
156 
157 
158 #     ------------------------------------------------------------
159 # STANDALONE MAIN LOOP (NON-ROS2)
160 #     ------------------------------------------------------------
161 def run_standalone():
162     """
163     Standalone mode without ROS2.
164     """
165     print("[Standalone] Starting non-ROS2 robot test...")
```

Code from MR.py XI

```
166
167     cam = init_camera()
168     motor = MotorDriver()
169     odo = Odometry()
170
171     motor_calibration(motor)
172     test_motors(motor)
173     test_camera(cam)
174
175     print("[Standalone] Driving slowly forward with odometry
         update...")
176     motor.set_speeds(50.0, 50.0)
177
178     t_prev = time.time()
179     for _ in range(20):
180         t_now = time.time()
181         dt = t_now - t_prev
182         t_prev = t_now
183
```

Code from MR.py XII

```
184         # Assume constant wheel angular velocity for demonstration
185         wl = 4.0
186         wr = 4.0
187         x, y, th = odo.update(wl, wr, dt)
188         print(f"[Odom] x={x:.3f} m, y={y:.3f} m, theta={th:.3f} rad")
189         time.sleep(0.1)
190
191     motor.stop()
192     print("[Standalone] Done.")
193
194
195 # ----------------------------------------------------------------------
196 # ROS2 NODE
197 # ----------------------------------------------------------------------
```

Code from MR.py XIII

```
198 class MobileRobotNode(Node):
199     """
200     ROS2 node controlling the mobile robot.
201 
202     Subscribes:
203       - /cmd_vel (geometry_msgs/Twist)
204 
205     Publishes:
206       - /odom (nav_msgs/Odometry)
207     """
208 
209     def __init__(self):
210         super().__init__("mobile_robot_node")
211         self.motor = MotorDriver()
212         self.odo = Odometry()
213 
214         self.last_time = self.get_clock().now().nanoseconds
    / 1e9
215         self.current_cmd = Twist()
216 
```

Code from MR.py XIV

```
217         self.cmd_sub = self.create_subscription(
218             Twist, "/cmd_vel", self.cmd_vel_callback, 10
219         )
220         self.odom_pub = self.create_publisher(
221             Odometry, "/odom", 10
222         )
223 
224         self.timer = self.create_timer(0.05, self.
    update_loop)
225         self.get_logger().info("MobileRobotNode started.")
226 
227     def cmd_vel_callback(self, msg):
228         """
229         Handle incoming velocity commands.
230         """
231         self.current_cmd = msg
232 
233     def update_loop(self):
234         """
235         Periodic update: odometry + motor command.
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Code from MR.py XV

```
236         """
237         now = self.get_clock().now().nanoseconds / 1e9
238         dt = now - self.last_time
239         self.last_time = now
240         if dt <= 0.0:
241             return
242 
243         v = self.current_cmd.linear.x
244         w = self.current_cmd.angular.z
245 
246         # Convert v, w to wheel angular velocities
247         wl = (2.0 * v - w * self.odo.L) / (2.0 * self.odo.r)
248         wr = (2.0 * v + w * self.odo.L) / (2.0 * self.odo.r)
249 
250         # Update odometry
251         x, y, theta = self.odo.update(wl, wr, dt)
252 
253         # Simple mapping from desired v to motor "speed"
    units
254         left_cmd = max(min(v * 100.0, 100.0), -100.0)
```

Code from MR.py XVI

```
255         right_cmd = max(min(v * 100.0, 100.0), -100.0)
256         self.motor.set_speeds(left_cmd, right_cmd)
257 
258         # Publish odometry
259         odom_msg = Odometry()
260         odom_msg.header.frame_id = "odom"
261         odom_msg.child_frame_id = "base_link"
262 
263         stamp = self.get_clock().now().to_msg()
264         odom_msg.header.stamp = stamp
265 
266         odom_msg.pose.pose.position.x = float(x)
267         odom_msg.pose.pose.position.y = float(y)
268         odom_msg.pose.pose.position.z = 0.0
269 
270         # Orientation as yaw-only quaternion
271         qz = math.sin(theta / 2.0)
272         qw = math.cos(theta / 2.0)
273         odom_msg.pose.pose.orientation.z = float(qz)
274         odom_msg.pose.pose.orientation.w = float(qw)
```

Code from MR.py XVII

```
275
276             self.odom_pub.publish(odom_msg)
277
278
279 def run_ros2():
280     """
281     Run the robot as a ROS2 node.
282     """
283     if not ROS2_AVAILABLE:
284         print("[ROS2] rclpy or messages not available.
    Install ROS2 first.")
285         return
286
287     rclpy.init()
288     node = MobileRobotNode()
289     try:
290         rclpy.spin(node)
291     except KeyboardInterrupt:
292         pass
293     finally:
```

Code from MR.py XVIII

```
294             node.motor.stop()
295             node.destroy_node()
296             rclpy.shutdown()
297             print("[ROS2] Node shut down.")
298
299
300 #
        ------------------------------------------------------------
301 # ENTRY POINT
302 #
        ------------------------------------------------------------
303 if __name__ == "__main__":
304     # Choose whether to run standalone or ROS2 mode.
305     # For a real project you could parse command line
        arguments.
306     USE_ROS2 = False
307
308     if USE_ROS2:
```

Code from MR.py XIX

```
309         run_ros2()
310     else:
311         run_standalone()
```

Listing 1: Mobile Robot Code

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Vielen Dank

für Ihre Aufmerksamkeit

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Quellen

Quellen I

Nachfolgend werden die Quellen der Bilder angegeben, die für diese Präsentation in ihrer ursprünglichen Form oder modifiziert verwendet worden sind.